

Pet Territory Wars — API Design v1.0

Backend: Go

Mobile: Flutter

API Style: REST + JSON

Base Path: /api/v1

Walk Processing: Asenkron

Result Delivery: Polling

Scope: MVP

1. Amaç

Bu doküman Flutter mobil uygulaması ile Go backend arasındaki API sözleşmesinin temel kurallarını tanımlar.

MVP API'sinin ana görevleri:

- Oyuncu ve pet profilini sunmak
- Walk başlatmak
- GPS noktalarını batch olarak almak
- Walk'ı tamamlayıp işleme kuyruğına eklemek
- Walk durumunu ve sonucunu sunmak
- Görünür harita alanındaki territory hücrelerini döndürmek
- Şehir bazlı leaderboard sunmak
- Tekrarlanan istekleri güvenli şekilde yönetmek

API iş kurallarını hesaplamaz.

```
HTTP Request
  ↓
API Layer
  ↓
Application Use Case
  ↓
Game Engine
  ↓
Application Result
  ↓
API Response
```

2. Temel API Prensipleri

API şu prensiplere dayanır:

```
REST tabanlı
Stateless
Versiyonlu
Idempotent yazma işlemleri
Cursor tabanlı pagination
Tutarlı hata formatı
Asenkron Walk processing
Viewport bazlı territory okuma
Domain ve API model ayrımı
```

MVP'de kullanılmayacaklar:

```
GraphQL
WebSocket
Push tabanlı Walk sonucu
gRPC public mobile API
Generic query endpoint
Client tarafında authoritative business rule
```

3. Base URL ve Versioning

Bütün endpoint'ler şu path altında bulunur:

```
/api/v1
```

Örnek:

```
POST /api/v1/walks
```

Version yalnızca path içinde tutulur.

Header tabanlı versioning kullanılmaz.

Breaking değişiklik gerektiğinde:

```
/api/v2
```

oluşturulur.

Yeni optional alan eklemek breaking change değildir.

Alan kaldırmak, tür değiştirmek veya anlamını değiştirmek breaking change'dir.

4. Content Type ve Encoding

Request ve response formatı:

```
Content-Type: application/json
Accept: application/json
```

Metin encoding:

```
UTF-8
```

Büyük GPS batch payload'ları için istemci şu header'ı kullanabilir:

```
Content-Encoding: gzip
```

Backend sıkıştırılmış body desteğini API Reference aşamasında uygulayacaktır.

5. Authentication

MVP API'si bearer token ile korunur.

```
Authorization: Bearer <access_token>
```

Backend token'dan authenticated player kimliğini çözer.

Client request body içinde authoritative `playerId` göndermemelidir.

Yanlış:

```
{
  "playerId": "... "
}
```

Doğru:

```
Player kimliği access token'dan alınır.
```

MVP authentication sağlayıcıları:

Google Sign-In
Apple Sign-In

Authentication akışının ayrıntıları ayrı security aşamasında netleştirilecektir.

6. Kimlik ve Veri Tipleri

6.1 UUID

Public entity ID'leri JSON içinde string olarak gönderilir.

```
{  
  "walkId": "019b7b93-7f23-7a1d-8a18-18b5575c1f11"  
}
```

6.2 H3 ID

Database'de `BIGINT` tutulur.

JSON'da string olarak taşınır.

```
{  
  "hexId": "622236750694711295"  
}
```

Sebebi:

- Dart ve JavaScript tarafında 64-bit integer taşınması sırasında hassasiyet kaybını önlemek
- JSON number limitlerinden bağımsız olmak
- Database storage tipini transport formatından ayırmak

6.3 Timestamp

Bütün zamanlar ISO 8601 ve UTC olarak gönderilir.

```
{  
  "createdAt": "2026-07-12T19:30:00Z"  
}
```

Timezone offset içeren zaman kabul edilebilir; backend UTC'ye normalize eder.

6.4 Ölçü Birimleri

Alan adında birim açıkça belirtilir.

```
distanceMeters
durationSeconds
accuracyMeters
speedMps
```

7. Response Envelope

Tek kaynak response'larda gereksiz ortak `data` envelope kullanılmayacaktır.

Başarılı response doğrudan kaynağı döndürür:

```
{
  "id": "019b...",
  "status": "ACTIVE"
}
```

Liste response'larında:

```
{
  "items": [],
  "nextCursor": null
}
```

kullanılır.

Hata response'u ortak formattadır.

8. Error Model

Bütün API hataları aşağıdaki yapıyı kullanır:

```
{
  "error": {
    "code": "WALK_ALREADY_FINISHED",
    "message": "This walk has already been finished.",
    "traceId": "4f2a6c8f0d8e4f7a",
    "details": {}
  }
}
```

```
}  
}
```

8.1 Alanlar

code

Machine-readable sabit hata kodudur.

Flutter uygulaması karar verirken `message` değil `code` kullanır.

message

Kullanıcıya doğrudan gösterilmesi zorunlu olmayan İngilizce teknik mesajdır.

UI localization Flutter tarafında hata koduna göre yapılabilir.

traceId

Log ve tracing korelasyonu içindir.

details

Alan bazlı validation veya ek güvenli bağlam içerir.

Örnek:

```
{  
  "error": {  
    "code": "VALIDATION_FAILED",  
    "message": "Request validation failed.",  
    "traceId": "a23f...",  
    "details": {  
      "fields": [  
        {  
          "field": "points",  
          "reason": "TOO_MANY_ITEMS"  
        }  
      ]  
    }  
  }  
}
```

9. HTTP Status Kodları

Status	Kullanım
200 OK	Başarılı okuma veya idempotent tekrar sonucu
201 Created	Yeni resource oluşturuldu
202 Accepted	İş kabul edildi, asenkron işlenecek
204 No Content	Başarılı fakat body gerekmiyor
400 Bad Request	Geçersiz JSON veya teknik request hatası
401 Unauthorized	Token eksik veya geçersiz
403 Forbidden	Kaynağa erişim yetkisi yok
404 Not Found	Resource bulunamadı
409 Conflict	State veya idempotency çakışması
413 Payload Too Large	Payload limiti aşıldı
422 Unprocessable Entity	Request teknik olarak doğru, domain açısından işlenemiyor
429 Too Many Requests	Rate limit aşıldı
500 Internal Server Error	Beklenmeyen sistem hatası
503 Service Unavailable	Geçici altyapı sorunu

Business validation sonucunda Walk'ın reddedilmesi, finish endpoint'inin HTTP hatası değildir.

Walk başarıyla kabul edilir, daha sonra:

```
status = REJECTED
```

olabilir.

10. Idempotency

Ağ sorunları nedeniyle mobil uygulama aynı isteği tekrar gönderebilir.

Aşağıdaki endpoint'lerde `Idempotency-Key` header zorunludur:

```
POST /walks
POST /walks/{walkId}/point-batches
POST /walks/{walkId}/finish
```

Örnek:

```
Idempotency-Key: 019b7b93-7f23-7a1d-8a18-18b5575c1f11
```

10.1 Kurallar

- Aynı kullanıcı, aynı endpoint ve aynı key ile aynı body'yi gönderirse önceki sonuç döner.
- Aynı key farklı body ile kullanılırsa `409 Conflict` döner.
- Key kullanıcı kapsamında değerlendirilir.
- Idempotency kaydı belirli retention süresi boyunca tutulur.
- Walk ID tek başına bütün endpoint'ler için yeterli idempotency garantisi değildir.

10.2 Batch Idempotency

GPS batch için ayrıca sıralı batch numarası kullanılır.

```
{  
  "batchSequence": 3,  
  "points": []  
}
```

Unique mantık:

```
walkId + batchSequence
```

Aynı batch tekrar gönderilirse ikinci kez point oluşturulmaz.

11. Walk API Akışı

MVP Walk akışı:

1. Walk oluştur
2. GPS batch'lerini gönder
3. Walk'ı finish et
4. Backend 202 döner
5. Worker işler
6. Flutter status endpoint'ini poll eder
7. Sonuç hazır olduğunda gösterir

Temel endpoint'ler:

```
POST /api/v1/walks  
POST /api/v1/walks/{walkId}/point-batches
```

```
POST /api/v1/walks/{walkId}/finish
GET /api/v1/walks/{walkId}
GET /api/v1/walks/{walkId}/result
GET /api/v1/walks
```

12. Walk Oluşturma

```
POST /api/v1/walks
```

Amaç:

- Yeni Walk lifecycle başlatmak
- Aktif şehir ve pet bilgisini ilişkilendirmek
- Flutter'a Walk ID döndürmek

Request:

```
{
  "petId": "019b...",
  "cityId": "019a...",
  "startedAt": "2026-07-12T19:00:00Z"
}
```

Response:

```
201 Created
```

```
{
  "walkId": "019c...",
  "status": "ACTIVE",
  "startedAt": "2026-07-12T19:00:00Z"
}
```

Kurallar:

- Player token'dan alınır.
- Pet oyuncuya ait olmalıdır.
- City aktif olmalıdır.
- Oyuncunun başka bir aktif Walk'ı bulunmamalıdır.
- Endpoint idempotent olmalıdır.

13. GPS Batch Upload

```
POST /api/v1/walks/{walkId}/point-batches
```

Tek tek GPS point endpoint'i kullanılmaz.

Amaç:

- Network çağrı sayısını azaltmak
- Database insert işlemlerini batch yapmak
- Offline kısa kesintilerde tekrar göndermeyi kolaylaştırmak

Request:

```
{
  "batchSequence": 4,
  "points": [
    {
      "sequence": 120,
      "recordedAt": "2026-07-12T19:14:03Z",
      "latitude": 41.015137,
      "longitude": 28.97953,
      "accuracyMeters": 7.4,
      "altitudeMeters": 38.2,
      "speedMps": 1.3,
      "isMockLocation": false
    }
  ]
}
```

Response:

```
200 OK
```

```
{
  "walkId": "019c...",
  "batchSequence": 4,
  "acceptedPointCount": 25,
  "lastAcceptedSequence": 144
}
```

13.1 Batch Kuralları

- Point sequence Walk boyunca artmalıdır.
- Aynı sequence ikinci kez farklı veriyle gönderilemez.

- Batch sequence tekrar gönderilebilir.
- Point timestamp sıralaması doğrudur.
- Teknik olarak geçersiz coordinate reddedilir.
- Business validation finish sonrasında engine tarafından yapılır.

13.2 Başlangıç Payload Limitleri

MVP başlangıç önerisi:

```
Minimum batch: 1 point
Maximum batch: 100 points
Maximum request body: 256 KB
Maximum points per Walk: 10.000
```

Bu değerler config içinde tutulur.

Calculator Rule değildir; API koruma limitidir.

14. Walk Finish

```
POST /api/v1/walks/{walkId}/finish
```

Amaç:

- Mobilin point upload işlemini tamamladığını bildirmek
- Walk'ı SUBMITTED durumuna geçirmek
- Processing job oluşturmak

Request:

```
{
  "endedAt": "2026-07-12T19:32:40Z",
  "lastPointSequence": 318
}
```

Response:

```
202 Accepted
```

```
{
  "walkId": "019c...",
  "status": "SUBMITTED",
  "submittedAt": "2026-07-12T19:32:43Z",
}
```

```
"statusUrl": "/api/v1/walks/019c..."
}
```

Kurallar:

- Walk authenticated player'a ait olmalıdır.
- Walk **ACTIVE** durumda olmalıdır.
- **lastPointSequence** , backend'in aldığı son sequence ile uyumlu olmalıdır.
- Eksik batch varsa **409 WALK_POINTS_INCOMPLETE** döner.
- Aynı finish isteği tekrar gelirse mevcut Walk durumu döner.
- Finish request'i engine hesabını beklemez.

15. Walk Durumu

```
GET /api/v1/walks/{walkId}
```

Amaç:

- Flutter polling akışını desteklemek
- Walk lifecycle durumunu göstermek

Processing response:

```
{
  "walkId": "019c...",
  "status": "PROCESSING",
  "startedAt": "2026-07-12T19:00:00Z",
  "endedAt": "2026-07-12T19:32:40Z",
  "submittedAt": "2026-07-12T19:32:43Z",
  "resolvedAt": null
}
```

Resolved response:

```
{
  "walkId": "019c...",
  "status": "RESOLVED",
  "validationStatus": "VALID",
  "startedAt": "2026-07-12T19:00:00Z",
  "endedAt": "2026-07-12T19:32:40Z",
  "resolvedAt": "2026-07-12T19:32:46Z"
}
```

Rejected response:

```
{
  "walkId": "019c...",
  "status": "REJECTED",
  "validationStatus": "INVALID",
  "rejectionReasons": [
    "TOO_SHORT_DISTANCE"
  ]
}
```

Flutter `PROCESSING` veya `SUBMITTED` durumunda polling yapmaya devam eder.

16. Walk Result

```
GET /api/v1/walks/{walkId}/result
```

Walk tamamlanmadıysa:

```
409 Conflict
```

```
{
  "error": {
    "code": "WALK_RESULT_NOT_READY",
    "message": "Walk result is not ready.",
    "traceId": "..."
  }
}
```

Resolved response:

```
{
  "walkId": "019c...",
  "status": "RESOLVED",
  "validationStatus": "VALID",
  "summary": {
    "totalDistanceMeters": 2348,
    "validDistanceMeters": 2217,
    "durationSeconds": 1960,
    "capturedHexCount": 12,
    "stolenHexCount": 1,
    "attackedHexCount": 3,
    "defendedHexCount": 4
  },
  "hexChanges": [
    {
```

```
{
  "hexId": "622236750694711295",
  "action": "EMPTY_CAPTURE",
  "dominanceBefore": 0,
  "dominanceAfter": 60
},
"ruleSetVersion": "mvp-0.1.0",
"engineVersion": "1.0.0",
"resolvedAt": "2026-07-12T19:32:46Z"
}
```

MVP'de response yalnızca bu Walk'ın etkilediği hex'leri döndürür.

Bütün territory state'i dönmez.

17. Walk Listesi

```
GET /api/v1/walks?cursor=...&limit=20
```

Amaç:

- Kullanıcının geçmiş Walk'larını listelemek

Response:

```
{
  "items": [
    {
      "walkId": "019c...",
      "status": "RESOLVED",
      "startedAt": "2026-07-12T19:00:00Z",
      "durationSeconds": 1960,
      "validDistanceMeters": 2217,
      "capturedHexCount": 12
    }
  ],
  "nextCursor": "eyJjcmVhdGVkQXQiOi..."
}
```

Limit:

```
Default: 20
Maximum: 50
```

Offset pagination kullanılmaz.

18. Territory API

MVP'de territory sorgusu bütün şehir verisini dönmaz.

Endpoint:

```
GET /api/v1/cities/{cityId}/territories
```

Query parametreleri:

```
north  
south  
east  
west  
resolution  
updatedAfter
```

Örnek:

```
GET /api/v1/cities/{cityId}/territories  
?north=41.02  
&south=41.00  
&east=29.00  
&west=28.96  
&resolution=9
```

Backend:

1. Viewport'u doğrular.
2. Viewport içindeki H3 cell listesini hesaplar.
3. `territory_hexes` tablosundan ilgili cell'leri toplu okur.
4. Sahipsiz ve database'de olmayan hücreleri varsayılan state olarak değerlendirebilir.
5. Kompakt response üretir.

Response:

```
{  
  "cityId": "019a...",  
  "resolution": 9,  
  "generatedAt": "2026-07-12T19:40:00Z",  
  "cells": [  
    {  
      "hexId": "622236750694711295",  
      "owner": {  
        "playerId": "019b...",  

```

```
    "displayName": "Luna",
  },
  "dominance": 85,
  "state": "OWNED",
  "updatedAt": "2026-07-12T19:35:13Z"
}
]
```

18.1 Viewport Limitleri

Backend aşırı büyük viewport'u reddetmelidir.

Başlangıç önerisi:

Maximum returned cells: 2.000

Aşılırsa:

VIEWPORT_TOO_LARGE

hatası döner.

Mobil uygulama zoom seviyesine göre daha dar sorgu yapar.

18.2 updatedAfter

Flutter daha önce aldığı harita state'inden sonra değişen hücreleri isteyebilir.

`GET /territories?...&updatedAfter=2026-07-12T19:30:00Z`

Bu incremental refresh için kullanılır.

Silinen veya sahihsiz hale gelen hücreler için ileride tombstone yaklaşımı gerekebilir.

MVP dominance decay doğrudan ownership silmediği için bu ihtiyaç sınırlıdır.

19. Leaderboard API

Endpoint:

`GET /api/v1/cities/{cityId}/leaderboard`

Query:

```
seasonId
cursor
limit
```

Aktif sezon belirtilmezse aktif sezon kullanılır.

Response:

```
{
  "season": {
    "seasonId": "019d...",
    "number": 8,
    "startsAt": "2026-07-06T00:00:00Z",
    "endsAt": "2026-07-13T00:00:00Z"
  },
  "items": [
    {
      "rank": 1,
      "playerId": "019b...",
      "displayName": "Luna",
      "activeHexCount": 42,
      "stealCount": 8,
      "defenseCount": 15
    }
  ],
  "nextCursor": null
}
```

Limit:

```
Default: 50
Maximum: 100
```

Leaderboard birkaç saniye gecikmeli olabilir.

Bu endpoint eventual consistency kabul eder.

20. Current Player Rank

Kullanıcı ilk 100 içinde değilse kendi sırasını ayrıca görebilmelidir.

Endpoint:

```
GET /api/v1/cities/{cityId}/leaderboard/me
```

Response:

```
{
  "rank": 147,
  "playerId": "019b...",
  "activeHexCount": 8,
  "stealCount": 1,
  "defenseCount": 3
}
```

Bu, bütün leaderboard'u indirip mobilde arama ihtiyacını kaldırır.

21. Player API

Current Player

```
GET /api/v1/me
```

Response:

```
{
  "playerId": "019b...",
  "displayName": "Luna",
  "homeCityId": "019a...",
  "status": "ACTIVE",
  "createdAt": "2026-07-01T10:00:00Z"
}
```

Player Stats

```
GET /api/v1/me/stats
```

Response:

```
{
  "currentSeason": {
    "activeHexCount": 12,
    "captureCount": 20,
    "stealCount": 4,
    "defenseCount": 9
  }
}
```

```
    },  
    "lifetime": {  
      "captureCount": 184,  
      "stealCount": 28,  
      "validWalkCount": 46,  
      "totalDistanceMeters": 126400,  
      "alphaWinCount": 2  
    }  
  }  
}
```

MVP'de diğer kullanıcıların ayrıntılı profili zorunlu değildir.

Harita ve leaderboard yalnızca güvenli özet profil bilgisi döndürür.

22. Pet API

MVP endpoint'leri:

```
POST /api/v1/pets  
GET /api/v1/pets  
GET /api/v1/pets/{petId}  
PATCH /api/v1/pets/{petId}
```

Pet MVP modeli:

```
{  
  "petId": "019e...",  
  "name": "Luna",  
  "status": "ACTIVE"  
}
```

Breed ve gelişim alanları MVP'de bulunmaz.

Bir oyuncunun tek aktif pet kuralı varsa backend bunu uygular.

23. City API

Şehir Listesi

```
GET /api/v1/cities
```

Response:

```
{
  "items": [
    {
      "cityId": "019a...",
      "code": "istanbul",
      "name": "İstanbul",
      "countryCode": "TR",
      "timezone": "Europe/Istanbul",
      "status": "ACTIVE"
    }
  ]
}
```

Şehir Detayı

```
GET /api/v1/cities/{cityId}
```

Şehir boundary polygon'u varsayılan response içinde dönmez.

Mobil harita için gerekli minimum şehir bilgileri döner.

24. Polling Stratejisi

MVP'de Walk sonucu polling ile alınır.

Önerilen Flutter davranışı:

```
İlk 10 saniye:
2 saniyede bir

10-30 saniye:
5 saniyede bir

30 saniye sonrası:
10 saniyede bir
```

Uygulama arka plana giderse agresif polling durdurulur.

Polling endpoint'i:

```
GET /api/v1/walks/{walkId}
```

Result yalnızca status tamamlandığında alınır.

API response şu header'ı verebilir:

```
Retry-After: 2
```

Flutter bu değeri kullanabilir.

25. Pagination

Büyük liste endpoint'lerinde cursor pagination kullanılır.

Cursor opaque string olmalıdır.

Client cursor içeriğini yorumlamaz.

Request:

```
GET /walks?cursor=abc&limit=20
```

Response:

```
{
  "items": [],
  "nextCursor": "def"
}
```

Cursor şu değerlere dayanabilir:

```
created_at + id
score fields + player_id
```

Offset kullanılmama nedeni:

- Büyük tablolarda performans kaybı
 - Yeni kayıtlar geldiğinde kayma ve duplicate riski
 - Leaderboard sıralamasında tutarsız sayfalar
-

26. Rate Limiting

Rate limit kullanıcı, IP ve endpoint türüne göre uygulanabilir.

MVP başlangıç limitleri:

Endpoint grubu	Önerilen limit
Authentication	10/dakika/IP
Walk oluşturma	5/dakika/player
Point batch upload	60/dakika/player
Walk finish	10/dakika/player
Walk status	60/dakika/player
Territory viewport	60/dakika/player
Leaderboard	30/dakika/player
Profile	30/dakika/player

Bu değerler config'tir ve load test sonrası değiştirilebilir.

Rate limit response:

```
429 Too Many Requests
Retry-After: 15
```

27. Request Limitleri

API genel güvenlik limitleri:

```
Maximum JSON depth
Maximum body size
Maximum array length
Maximum string length
Maximum query parameter count
Maximum viewport area
Maximum pagination limit
```

Limitler API katmanında uygulanır.

Büyük body parse edilmeden önce HTTP server seviyesinde sınırlandırılır.

28. Concurrency ve Conflict

API aynı hex üzerindeki concurrency'yi doğrudan çözmez.

Akış:

```
Finish request
→ Job
→ Engine calculation
→ Optimistic database update
```

Conflict uygulama/worker katmanında retry edilir.

Client'a geçici olarak yansıtılması gerekiyorsa:

```
WALK_PROCESSING_CONFLICT
```

gibi internal durum kullanılabilir.

Normal optimistic conflict mobil kullanıcıya doğrudan gösterilmemelidir.

29. Caching

MVP'de cache zorunlu değildir.

Cache uygulanabilecek endpoint'ler:

```
GET /cities
GET /cities/{cityId}
GET /cities/{cityId}/leaderboard
GET /cities/{cityId}/territories
```

Cache source of truth değildir.

Private endpoint'ler oyuncu bazlı cache key kullanır.

Walk status endpoint'i agresif ve uzun süreli cache edilmez.

HTTP cache header'ları territory ve city endpoint'lerinde kullanılabilir:

```
ETag
If-None-Match
Cache-Control
```

MVP başlangıcında zorunlu değildir ancak API tasarımı uyumludur.

30. DTO Kuralları

API DTO'ları domain ve database modellerinden ayrılır.

Örnek:

```
type UploadPointRequest struct {  
    Sequence      int      `json:"sequence"`  
    RecordedAt    time.Time `json:"recordedAt"`  
    Latitude       float64  `json:"latitude"`  
    Longitude     float64  `json:"longitude"`  
    AccuracyMeters float32   `json:"accuracyMeters"`  
}
```

Domain'e mapper ile çevrilir.

API response'a aşağıdakiler sızdırılmaz:

```
Database version  
Internal row ID  
Job lock state  
Raw database error  
RuleSet'in gizli anti-cheat parametreleri  
Internal trace details
```

31. RuleSet'in Mobile'a Gönderilmesi

MVP'de bütün authoritative RuleSet mobile gönderilmez.

Flutter'ın UI için ihtiyaç duyduğu güvenli bilgiler ayrı config endpoint'inden dönebilir.

Örnek:

```
GET /api/v1/app-config
```

Response:

```
{  
  "walk": {  
    "maximumUploadBatchSize": 100,  
    "locationCollectionIntervalSeconds": 5  
  },  
  "map": {
```

```
    "h3Resolution": 9
  },
  "features": {
    "leaderboardEnabled": true
  }
}
```

Aşağıdaki değerler gizli kalabilir:

```
Anti-spoof thresholds
Exact attack formula
Threat detection ağırlıkları
Fraud risk kuralları
```

Client config authoritative değildir.

Backend client değerinden bağımsız doğrulama yapar.

32. API Güvenliği

MVP API'sinde minimum güvenlik gereksinimleri:

```
TLS zorunlu
Bearer token
Resource ownership kontrolü
Request size limit
Rate limiting
Parameter validation
Parameterized SQL
Idempotency kontrolü
Sensitive field filtering
Trace ID
Raw GPS loglamama
```

Client'ın gönderdiği aşağıdaki bilgiler güvenilir kabul edilmez:

```
speedMps
isMockLocation
distance
duration
cityId doğruluğu
point sequence bütünlüğü
```

Backend bunları doğrular veya yeniden hesaplar.

33. API Performans Kuralları

- Walk point'leri batch alınır.
- Batch insert kullanılır.
- Territory tek tek değil toplu H3 lookup ile okunur.
- Bütün şehir territory response olarak dönmez.
- Leaderboard pagination zorunludur.
- Response'larda gereksiz nested nesne bulunmaz.
- `SELECT *` kullanılmaz.
- Mobilin kullanmadığı kolonlar gönderilmez.
- Büyük listelerde compression desteklenir.
- İşleme hesabı HTTP request içinde yapılmaz.

34. MVP Endpoint Özeti

Authentication

```
POST /api/v1/auth/google
POST /api/v1/auth/apple
POST /api/v1/auth/refresh
```

Current Player

```
GET /api/v1/me
GET /api/v1/me/stats
```

Pets

```
POST /api/v1/pets
GET /api/v1/pets
GET /api/v1/pets/{petId}
PATCH /api/v1/pets/{petId}
```

Cities

```
GET /api/v1/cities
GET /api/v1/cities/{cityId}
```

Walks

```
POST /api/v1/walks
POST /api/v1/walks/{walkId}/point-batches
POST /api/v1/walks/{walkId}/finish
GET /api/v1/walks/{walkId}
GET /api/v1/walks/{walkId}/result
GET /api/v1/walks
```

```
Territory
-----
GET /api/v1/cities/{cityId}/territories

Leaderboard
-----
GET /api/v1/cities/{cityId}/leaderboard
GET /api/v1/cities/{cityId}/leaderboard/me

Configuration
-----
GET /api/v1/app-config
```

35. MVP Dışında Bırakılan API'ler

```
WebSocket
Push notification registration
Guild
Clan
Professional walker
Breed bonus
NPC management
Sponsor territory
Inventory
Coin
Booster
Chat
Social feed
Event battle
Admin panel public API
```

36. Definition of Done

API Design v1.0 aşağıdaki koşulları sağlamalıdır:

1. Bütün endpoint'ler /api/v1 altında.
2. Walk işlemi asenkron.
3. GPS point'ler batch yükleniyor.
4. Walk finish 202 Accepted döndürüyor.
5. Polling ile status izlenebiliyor.
6. Yazma endpoint'leri idempotent.
7. H3 ID JSON'da string.
8. Territory viewport bazlı okunuyor.
9. Bütün şehir verisi tek response'ta dönüyor.

10. Listeler cursor pagination kullanıyor.
11. Error formatı ortak.
12. Domain modelleri doğrudan serialize edilmiyor.
13. Rate ve payload limitleri tanımlı.
14. Raw GPS loglanmıyor.
15. Client business rule kaynağı değil.
16. Leaderboard eventual consistency kabul ediyor.
17. Same-player resource ownership kontrolü var.
18. API worker ve engine detaylarını client'a sızdırmıyor.

37. Nihai Karar

Pet Territory Wars MVP API'si:

```
REST
JSON
/api/v1
Bearer authentication
Batch GPS upload
Asenkron Walk resolve
Polling result
Cursor pagination
Viewport territory query
Idempotent writes
Tutarlı error codes
```

üzerine kurulacaktır.

Temel Walk sözleşmesi:

```
POST Walk
→ POST Point Batches
→ POST Finish
→ GET Status
→ GET Result
```

Bu yapı MVP için yeterince basit, mobil ağ problemlerine dayanıklı ve aynı anda binlerce Walk isteğinin kontrollü biçimde işlenmesine uygundur.